Probabilistic Graphical Models

Causal Bayesian Networks
Guidelines and Rubrics
for Presentation and Report

Semester 1, 2026–2027

Instructor: Nguyen Chi Thien, Ph.D.

Outline

1. Tutoring illustration: causal concepts and worked examples
2. UCI project: B0, H1 and H2
3. Shared test prediction and intervention comparisons
4. Standard evaluation criteria and reporting tables
5. Presentation and report rubrics (10 points each)
6. Detailed project guide (slides 53–80). Ablations are optional.

Teaching illustration: tutoring and exam success

Classroom example: how would exam success change if all students received tutoring?
T: tutoring participation (0=no, 1=yes). Y: exam result (0=fail, 1=pass).
Z: prior preparation (0=low, 1=high), measured before tutoring.
The supplied model and example code explain causal concepts and probability calculations.
The graded project uses UCI data and compares B0, H1 and H2. It does not require creating data or running a synthetic benchmark.

Tutoring vocabulary: variables, states and parents

A variable is a column describing each student. Its states are the possible values and their meanings.

T: tutoring participation, with states 0=no and 1=yes. Y: exam result, with states 0=fail and 1=pass.

Z: preparation measured before tutoring, with states 0=low and 1=high. Keep the same coding in every model and split.

A parent is a direct input to a node’s mechanism. In the teaching DAG, Y has parents T and Z, so its CPD is P(Y | T,Z).

An estimand is the precise effect we want: the increase in pass probability if the same target students all receive tutoring rather than none.

These binary definitions belong to the synthetic example. UCI support variables need their own timing and intervention justification.

Causal graph and assumptions

Graph: $Z \to T, \quad Z \to Y, \quad T \to Y$. Z is a common cause of T and Y.
Factorization: $P(Z, T, Y) = P(Z)\, P(T \mid Z)\, P(Y \mid T, Z)$.
Causal interpretation requires defensible directions and stable mechanisms under intervention.
Assume no unmeasured confounding, consistency and positivity: $0 < P(T = t \mid Z = z) < 1$.
A structure learned from observational data alone does not establish causal directions.

Tutoring DAG: paths and confounding

An edge states a direct causal influence assumed by the model. Z affects both tutoring participation T and passing Y. Tutoring T can affect Y.

A path is a sequence of connected variables. T to Y is the causal path whose total effect interests us.

The path from T back through Z to Y is a backdoor path. It can create an association even without a tutoring effect.

Z is a confounder here because it is a common cause of T and Y. Better-prepared students are more likely to take tutoring in this teaching model.

Therefore, the tutored and untutored groups have different starting preparation. Comparing their raw pass rates mixes selection with tutoring.

Adjusting for Z compares tutoring choices at a common preparation level before averaging over a common target group.

Tutoring assumptions: what each one means

No unmeasured confounding: after accounting for Z, assume no omitted common cause of tutoring and passing remains.

Example concern: motivation could affect both tutoring and exam success. If Z does not account for it, our causal interpretation can fail.

Consistency: “tutoring” must describe a sufficiently well-defined intervention. Record the program and avoid mixing incompatible versions silently.

Positivity or overlap: at each preparation level in the target group, both tutoring choices must have support in the training data.

In this synthetic model, tutoring probabilities are 0.2 for low preparation and 0.8 for high preparation. Both lie strictly between zero and one.

A DAG and an executable formula express these assumptions. Good test prediction or successful software identification does not verify them.

Tutoring extensions: mediator and collider

Confounder example: prior preparation Z affects both tutoring T and passing Y. A valid adjustment can remove this source of selection bias.

Mediator example: tutoring increases practice time M, and practice time improves passing. M carries part of the tutoring effect.

To estimate the total effect, let M respond to tutoring in the causal calculation. Holding M fixed changes the question and may remove part of the effect.

Collider example: a later remedial referral R depends on both tutoring participation and a student's underlying academic difficulty D.

Restricting analysis to referred students can make T and D associated. Since D affects passing, conditioning on R can introduce bias.

M, D and R are hypothetical extensions for explaining variable roles. They are not additional variables in the original three-node benchmark.

Sample conditional probability tables

Quantity	Condition	Probability
$P(Z = 1)$	Prior preparation	0.50
$P(T = 1 \mid Z)$	$Z = 0$ / $Z = 1$	0.20 / 0.80
$P(Y = 1 \mid T, Z)$	$T = 0,\ Z = 0$	0.10
$P(Y = 1 \mid T, Z)$	$T = 0,\ Z = 1$	0.50
$P(Y = 1 \mid T, Z)$	$T = 1,\ Z = 0$	0.40
$P(Y = 1 \mid T, Z)$	$T = 1,\ Z = 1$	0.80

For every binary variable, $P(\text{value} = 0 \mid \text{parents}) = 1 - P(\text{value} = 1 \mid \text{parents})$.

Tutoring CPD: learning a probability from counts

A CPD gives one node's probability distribution for each combination of its parents. For Y, we need pass/fail probabilities for every (T,Z) group.

Illustrative training group: among 10 low-preparation students without tutoring, 1 passes. The maximum likelihood pass estimate is 1/10 = 0.10.

If the estimated pass probability is 0.10, the fail probability for that same group is 0.90. The column must sum to one.

With few observations, smoothing adds prior pseudo-counts before normalization. Five passes out of five gives ML=1, but add-one smoothing gives 6/7.

Learning structure chooses the parents. Learning parameters fills the CPDs after the graph is chosen. pgmpy and bnlearn can help with these distinct tasks.

The count examples are illustrations. Save the actual training counts, fitted CPDs and prior settings in your experiment.

Observation and intervention

Observation: $P(Y = 1 \mid T = t) = \sum_z P\,(Y = 1 \mid t, z)P(z \mid t)$.
Intervention $\mathrm{do}(T = t)$ fixes T and removes the edge $Z \to T$.
Truncated factorization: $P\big(z, y \mid \mathrm{do}(T = t)\big) = P(z)P(y \mid t, z)$.
Therefore $P\big(Y = 1 \mid \mathrm{do}(T = t)\big) = \sum_z P\,(Y = 1 \mid t, z)P(z)$.
Risk difference (ATE for binary Y): $P\big(Y = 1 \mid \mathrm{do}(T = 1)\big) - P\big(Y = 1 \mid \mathrm{do}(T = 0)\big)$.

Tutoring observation versus intervention

Observation asks: among students who happened to take tutoring, how many pass? This is P(Y=1 | T=1).

Intervention asks: if we assigned tutoring to the target students, how many would pass? This is P(Y=1 | do(T=1)).

In the synthetic population, observed pass rates are 0.18 without tutoring and 0.72 with tutoring. Their difference is 0.54.

The intervention pass rates are 0.30 and 0.60. Their difference is 0.30, or 30 percentage points.

The raw groups differ in preparation. Intervention compares tutoring choices for a common preparation distribution.

Changing T in a predictor becomes a valid do calculation only when the identified formula or causal generative model justifies it.

Tutoring adjustment: why the set is {Z}

An adjustment set is a collection of observed variables used in an identified formula for the causal query. It is not another prediction model.

For the tutoring DAG, {Z} blocks the backdoor path through prior preparation. Compare T=0 and T=1 separately within each Z level.

Then average both tutoring choices using the same target preparation weights. This removes the difference in preparation mix between the raw groups.

Here, risk(t) = sum_z P(Y=1 | T=t,Z=z) times P_target(Z=z). Use test-cohort weights for the required common-reference comparison.

B0 and H1 share the graph and identified formula. If they use adjustment, they use the same Z and reference weights, changing only the selected CPD.

A changed H2 graph requires new identification. Some do queries use other formulas and do not require selecting a separate backdoor adjustment set.

Tutoring truncated factorization: replacing T’s rule

The original CBN uses P(Z,T,Y) = P(Z) P(T | Z) P(Y | T,Z). Each factor is one variable’s mechanism.

For do(T=1), stop drawing tutoring status from P(T | Z). Assign tutoring to everyone while retaining the other mechanisms.

The remaining distribution is P(Z,Y | do(T=1)) = P(Z) P(Y | T=1,Z). Summing over Z gives the intervention pass probability.

For the synthetic population: 0.5 times 0.40 plus 0.5 times 0.80 = 0.60 under tutoring.

To target a test cohort with 40% low and 60% high preparation, average the same conditional intervention risks using 0.4 and 0.6 instead.

Adjustment and truncated factorization agree for this graph. Truncated factorization extends the computation to graphs with more mechanisms.

Tutoring discovery, identify, estimate and refute

Discover or specify the graph: decide whether preparation affects tutoring and passing, using domain constraints and a declared learning method.

Identify: given the graph and assumptions, determine a valid expression for the effect. In this example, DoWhy can find adjustment by Z.

Estimate: use training data to fit the probabilities in that expression. B0 uses basic CPDs, while H1 uses a neural outcome CPD.

Compute the query: evaluate both tutoring choices for the same test preparation profiles and average the probabilities.

Refute: try a placebo treatment or data-subset experiment to inspect robustness. Record what estimator the check actually refits.

Identification yields a justified formula under assumptions. Estimation and query evaluation yield numerical results with uncertainty.

Exact reference values

Quantity	$T = 0$	$T = 1$	Difference $(1 - 0)$
$P(Z = 1 \mid T = t)$	0.20	0.80	0.60
$P(Y = 1 \mid T = t)$	0.18	0.72	0.54
$P\big(Y = 1 \mid \mathrm{do}(T = t)\big)$	0.30	0.60	0.30

Example: $P\big(Y = 1 \mid \mathrm{do}(T = 1)\big) = 0.40{\times}0.50 + 0.80{\times}0.50 = 0.60$. These are population values, not measured software outputs.

Tutoring test reference: a complete numerical example

Illustrative reference cohort: 100 test students, with 40 at low preparation and 60 at high preparation. Freeze this cohort for all models.

Use the teaching conditional risks: without tutoring, 0.10 for low preparation and 0.50 for high. With tutoring, 0.40 and 0.80.

No tutoring: risk(0) = 0.4 times 0.10 plus 0.6 times 0.50 = 0.34.

Tutoring for everyone: risk(1) = 0.4 times 0.40 plus 0.6 times 0.80 = 0.64.

The treatment contrast is 0.64 minus 0.34 = 0.30. ATE is the average change in pass probability, here 30 percentage points.

The test Z values supply the weights. Test Y labels are not used in this calculation. These are known-mechanism illustrations, not fitted model results.

Classroom illustration: tutoring notebook workflow

This classroom walkthrough explains how the supplied tutoring model works. It is not a project deliverable.

1. Inspect the defined variables, graph and example conditional probabilities.
2. Fit the teaching BN, compare observation with intervention and inspect the identification formula.
3. Optionally run the example data-generation and GCM cells to understand the mechanics.
4. Interpret the reference values as properties of the specified illustration.

For the project, proceed directly to the UCI comparison criteria on slides 28, 37, 69 and 74–75.

Classroom code: illustrative data generation

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(42)
n = 20000
z = rng.binomial(1, 0.5, n)
t = rng.binomial(1, 0.2 + 0.6*z)
y = rng.binomial(1, 0.1 + 0.3*t + 0.4*z)
df = pd.DataFrame({"Z": z, "T": t, "Y": y})
edges = [("Z", "T"), ("Z", "Y"), ("T", "Y")]
print(df.groupby(["Z", "T"]).size())
```

Each draw uses fresh random noise. The generating probabilities match the reference CPDs.

Sample code: BN fitting and do query

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import BayesianEstimator
from pgmpy.inference import VariableElimination
bn = DiscreteBayesianNetwork(edges)
bn.fit(df, estimator=BayesianEstimator, prior_type="BDeu",
       equivalent_sample_size=5,
       state_names={v: [0, 1] for v in df.columns})
assert bn.check_model()
q = VariableElimination(bn)
pz = q.query(["Z"]).values
def p_do(t):
    return sum(q.query(["Y"], evidence={"T": t, "Z": z})
               .values[1] * pz[z] for z in [0, 1])
print(p_do(0), p_do(1), p_do(1) - p_do(0))
```

This adjustment implements $\mathrm{do}(T = t)$ for the sample DAG. Querying evidence $T = t$ alone computes observation.

Sample code: DoWhy effect estimation

```
from dowhy import CausalModel
model = CausalModel(
    data=df, treatment="T", outcome="Y",
    graph="digraph { Z -> T; Z -> Y; T -> Y; }")
estimand = model.identify_effect(
    proceed_when_unidentifiable=False)
print(estimand)
estimate = model.estimate_effect(
    estimand, method_name="backdoor.linear_regression",
    control_value=0, treatment_value=1,
    target_units="ate", confidence_intervals=True)
print(estimate.value)
print(estimate.get_confidence_intervals())
```

DoWhy should identify adjustment for Z. The additive outcome probability makes linear adjustment suitable here.

Sample code: $\mathrm{do}(T = t)$ in DoWhy GCM

```
import networkx as nx
from dowhy import gcm
cat = df.astype(str)  # categorical states "0", "1"
cm = gcm.ProbabilisticCausalModel(nx.DiGraph(edges))
gcm.auto.assign_causal_mechanisms(cm, cat)
gcm.fit(cm, cat)
np.random.seed(42)
risks = []
for t in [0, 1]:
    s = gcm.interventional_samples(
        cm, {"T": lambda x, t=t: str(t)},
        num_samples_to_draw=20000)
    assert (s["T"] == str(t)).all()
    risks.append((s["Y"] == "1").mean())
print(risks, risks[1] - risks[0])
```

The constant function sets T for every sample. Results include mechanism fitting error and Monte Carlo error.

Sample code: refutation checks

```
placebo = model.refute_estimate(
    estimand, estimate,
    method_name="placebo_treatment_refuter",
    placebo_type="permute", num_simulations=100,
    random_seed=42)
subset = model.refute_estimate(
    estimand, estimate,
    method_name="data_subset_refuter",
    subset_fraction=0.8, num_simulations=100,
    random_seed=42)
print(placebo)
print(subset)
```

Expected behavior: placebo effect near 0, subset effect near the original estimate. Report actual outputs and test definitions.

Verification of $\mathrm{do}(T = t)$

Implementation check: compare BN adjustment with the exact values 0.30, 0.60 and $\mathrm{ATE} = 0.30$.
Independent estimate: compare the BN ATE and the DoWhy adjusted ATE on the same dataset and contrast.
Simulation check: use GCM to confirm every intervened T equals t and compare simulated risks with BN adjustment.
Robustness check: run at least two DoWhy refuters and discuss their expected behavior.
Uncertainty: repeat across seeds and report error, spread or confidence intervals. Passing refutation tests cannot prove causality.

UCI tabular data: required source

The main application must use a public, structured tabular dataset from the UCI Machine Learning Repository.

Record the dataset page, identifier or DOI, file version, download date and license.

Document the unit of observation, column meanings, outcome, treatment and measurement times.

Select a question with a defensible intervention and measured potential confounders. UCI membership alone does not justify a causal interpretation.

Use this UCI dataset for all required B0/H1/H2 experiments. The tutoring walkthrough is instructional and requires no separate project experiment.

UCI application example: Student Performance

Dataset: Student Performance, UCI ID 320. Use one subject file and report which one.

Question: Under stated assumptions, how would extra educational support affect the probability of passing?

Treatment: schoolsup (yes/no). Outcome: pass, defined for this project as final grade G3 of at least 10.

Candidate baseline variables: school, age, sex, parental education and previous failures. Justify their timing and role in the DAG.

G1 and G2 are earlier-period grades. Include them only if measured before the chosen intervention.

The dataset does not establish support timing or absence of hidden confounding. Present causal results conditional on these assumptions.

UCI example: reproducible data loading

```
from ucimlrepo import fetch_ucirepo
import pandas as pd
uci = fetch_ucirepo(id=320)
data = pd.concat([uci.data.features,
                  uci.data.targets], axis=1)
print(uci.metadata)
print(uci.variables)
data["support"] = data["schoolsup"].map(
    {"no": 0, "yes": 1})
data["pass"] = (data["G3"] >= 10).astype(int)
# Remove G3 from predictors after creating pass.
# Select pre-intervention covariates using the DAG.
```

Install ucimlrepo first. Check missing values, coding, subject metadata and treatment overlap before modeling.

Baseline and hybrid comparison protocol

Use common train/validation/test splits and at least five paired runs for B0, H1 and H2. Keep treatment and control values fixed.

TRAIN fits preprocessing, DAGs and CPDs or neural weights. VALIDATION selects settings and stopping. Freeze all choices before TEST evaluation.

On the same TEST rows, compare predictions with observed Y. For binary Y, report AUROC, macro-F1 and Brier score or log loss.

For causal comparison, use the same TEST baseline covariates as the reference population. Average each model's identified do-query over those rows.

Test Y labels are not used to calculate these intervention means. Re-identify when the DAG changes and check baseline timing and overlap.

Compare effects, uncertainty, stability, refutation and runtime. Core criteria and report fields appear on slides 69 and 74–75.

Tutoring TRAIN, VALIDATION and TEST in practice

Illustrative data size: 1,000 students split into 700 TRAIN, 150 VALIDATION and 150 TEST rows, with saved row IDs.

TRAIN: learn B0's graph and CPDs, H1's neural weights, and H2's neural graph. Fit any encoders or bins here too.

VALIDATION: select settings such as smoothing strength, hidden-layer size, graph threshold and neural stopping epoch.

TEST prediction: predict each student's passing outcome and score against the observed Y. Freeze every modeling choice first.

TEST causal reference: use each student's baseline Z to compute outcomes under both tutoring choices, then average. Y is not needed for this step.

All models share the same splits. Two models tested on different student groups could differ because of group composition rather than method.

H1: neural estimation and H2: neural DAG learning

B0: learn a DAG with a traditional method in pgmpy or bnlearn, using domain constraints. Fit non-neural CPDs. H1 keeps this DAG and replaces selected CPDs with neural models.

H2: train neural causal discovery, such as DAG-GNN, on the same training data with the same temporal and domain constraints as B0.

For H2, rerun identification and retain the B0 non-neural CPD fitting family and effect-computation procedure, adapting parent or adjustment sets.

Compare B0 vs H1 for neural mechanisms and B0 vs H2 for graph learning. Report identification failures and uncertain edge directions.

A combined H1+H2 model is optional. ChatGPT graph suggestions alone do not satisfy neural causal discovery.

Concrete implementation steps appear on slides 59–63. Optional ablations appear on slides 64–67.

Tutoring B0, H1 and H2: what changes?

B0: learn a tutoring DAG with a traditional algorithm in pgmpy or bnlearn, subject to timing constraints. Fit basic conditional probability tables.

H1: copy that exact B0 graph. Replace only the outcome mechanism P(Y | T,Z) with neural probabilities if those are Y's parents.

H2: use a neural method to learn the graph from the same training rows and domain constraints. Fit basic CPDs on its selected graph.

Example graph difference: a learner might omit the preparation-to-tutoring edge. Record that choice and rerun identification instead of assuming the old formula.

For all three, ask the same question about the same test students: how would passing change under tutoring versus no tutoring?

Compare prediction scores and standardized effects separately. A changed graph or a larger ATE does not automatically imply a better model.

Controlled differences between B0, H1 and H2

Component	B0 and H1	H2
DAG	B0: traditional discovery. H1: same learned DAG.	Neural discovery proposes a constrained DAG.
Identification	Same DAG and identified formula. Same Z if adjusting.	Re-identify under the proposed DAG.
CPD fitting	B0: non-neural. H1: selected neural CPDs.	Same non-neural fitting family as B0.
Comparison	B0 vs H1 isolates neural mechanisms.	B0 vs H2 isolates structure learning.

H1 keeps B0's identified formula, including its valid set when using adjustment. Tutoring example: slide 13.

Optional ablations: one change per variant

Variant	Single change from its parent model	Purpose
B0 / H1 / H2	Primary reference models. Compare H1/B0 and H2/B0.	Mechanism and structure contributions
A1 from H1	Remove dropout or one active neural penalty.	Effect of regularization
A2 from H2	Remove one forbidden-edge constraint. Retrain DAG learning.	Value of domain information
A3 from B0	Replace smoothed CPDs with maximum likelihood.	Sensitivity to sparse counts

A1–A3 are optional extensions, not required for full credit. The core comparison is B0/H1/H2. Optional procedures appear on slides 64–67.

Optional tutoring ablations: concrete examples

A1 starts from H1. If its outcome network uses dropout=0.2, set dropout=0 and keep layers, inputs and other settings unchanged.

A2 starts from H2. Remove one rule forbidding exam result Y from pointing to prior preparation Z, then retrain graph learning.

Removing that prohibition only allows a candidate edge. It does not force the edge to appear. Keep the acyclicity rule active.

A3 starts from B0. Keep its learned graph fixed and refit CPDs using maximum likelihood instead of smoothing.

If five of five students in a group pass, ML gives 1. An illustrative add-one estimate gives 6/7. Inspect how sparse cells affect the final query.

Compare A1 with H1, A2 with H2 and A3 with B0 on the same test cohort. Removing H1’s only neural CPD just returns B0.

Optional ablation experiment protocol

If choosing an optional ablation, define its parent, hypothesis and exactly one changed component. A1 pairs with H1, A2 with H2, and A3 with B0.

Use five or more paired runs. Keep eligible rows, split indices, seeds, preprocessing and unchanged settings fixed.

A1 removes one active regularizer. A2 removes one domain constraint while retaining acyclicity. A3 changes CPD smoothing to maximum likelihood.

Keep the same tuning and stopping protocol. Report actual runtime, convergence and any implementation failures.

For A2, rerun identification and flag implausible directions. On UCI data, report effect changes. No synthetic-data experiment is required.

Show parent and variant results with signed paired changes. Distinguish seed variability from sampling uncertainty. See slides 64–75.

Tutoring paired runs, spread and confidence intervals

A paired run uses the same train, validation and test row IDs for B0, H1 and H2. Each pair answers the same test-cohort question.

Illustrative pair: B0 ATE=0.28 and H1 ATE=0.31. With delta=H1 minus B0, the paired change is +0.03, or 3 percentage points.

Repeat at least five times and report all run values. Mean summarizes the typical estimate, while standard deviation summarizes run-to-run spread.

With a fixed split and changing neural seed, spread measures training instability. It is not automatically a confidence interval for the true tutoring effect.

For a bootstrap interval conditional on the fixed test cohort, resample and refit the training pipeline using a justified sampling scheme.

On UCI, compare prediction, intervention estimates and robustness. Discuss paired differences without treating them as error against unknown truth.

Required result tables and interpretation

Table A: held-out prediction for B0, H1 and H2. Report AUROC, macro-F1 and Brier score or log loss for binary Y, or MAE and RMSE for regression.

Table B: the same test-reference intervention means, ATE, 95% confidence interval and the declared uncertainty method.

Include the graph, identified formula or adjustment set, reference sample size and common treatment/control values.

Table C: paired-run stability, two DoWhy refutation checks, fit/query runtime and hardware. Optional ablations go in an appendix.

Compare models across these criteria. A larger ATE or narrower interval alone does not establish a better causal estimate.

Detailed criteria and table fields appear on slides 69 and 74–75. No synthetic-data result table is required.

Tutoring prediction metrics versus causal estimates

Prediction question: can the model forecast the observed pass/fail label of a held-out student from eligible measured inputs?

AUROC measures ranking across classification thresholds. Macro-F1 summarizes classwise precision/recall at a threshold chosen before test evaluation.

Brier score measures squared probability error. If the predicted pass probability is 0.8 and the student passes, this row contributes (0.8-1)^2 = 0.04.

Log loss penalizes assigning low probability to what actually happens. Report it or Brier score alongside ranking and classification metrics.

Causal question: how does average passing change if tutoring is assigned differently? Its outputs are intervention risks and ATE, not AUROC.

Use actual test Y for prediction metrics. Use baseline test covariates and the identified causal formula for the intervention comparison.

Presentation: required scope

Choose one UCI dataset and define timing, population, intervention contrast and estimand. See slides 54–58.
Implement B0 with traditional DAG learning and basic CPDs, H1 with neural CPDs on that DAG, and H2 with neural DAG learning. See slides 59–63.
Fit on TRAIN, select on VALIDATION and compare on the same TEST cohort. Use at least five paired runs.
Report prediction scores, standardized intervention estimates with uncertainty, stability, refutation and runtime. See slides 68–71 and 74–75.
Submit a reproducible notebook, presentation and report. The tutoring walkthrough is illustrative. No synthetic generation or benchmark is required.
Ablations and additional GCM simulations are optional. Full-credit requirements and report structure appear on slides 43–50 and 76–80.

UCI project selection: possible questions

Application	Possible treatment X	Outcome Y	Potential confounders
Education	Study support	Exam success	Prior scores, background
Business	Promotion exposure	Purchase	Prior demand, customer profile
Operations	Preventive maintenance	Failure	Machine age, prior condition
Public policy	Program participation	Employment	Prior income, education

Every application must use a UCI tabular dataset. Select measured variables and justify timing, intervention meaning and adjustment assumptions.

Libraries and their roles

ucimlrepo or official UCI files: obtain tabular data and preserve source metadata.
pgmpy: traditional DAG learning, CPD fitting and inference. Alternatively use bnlearn (R), such as hc or tabu, for traditional DAG learning.
For bnlearn, export named edges and node states to the Python pipeline, or document the R/Python bridge. Use identical train rows and constraints.
DoWhy (required): graph-based identification, supported estimation/refutation and refutation. GCM simulation is optional.
PyTorch or TensorFlow/Keras: H1 neural CPDs and H2 neural discovery. Keep raw variables and domain information comparable.
Record exact packages, language/runtime versions, seeds and hardware. Here bnlearn means the R package at bnlearn.com.

Presentation rubric: foundations (4 points)

Criterion	Evidence for full credit	Points / CLO
Causal representation	Variables, DAG, factorization and assumptions	1 / CLO1, CLO2
Learning and DL roles	Basic CPDs, neural extensions and identification limits	2 / CLO1, CLO2
Inference	Observation, identification and valid intervention queries	1 / CLO1, CLO2

Foundations total: 4 points. Award partial credit for partially correct evidence.

Presentation rubric: application (6 points)

Criterion	Evidence for full credit	Points
Problem and UCI data	Dataset source and defensible causal question	1
Implementation	Runnable B0, H1 and H2 with explicit DL integration	1
Prediction comparison	Shared TEST metrics and paired differences	1
Intervention comparison	Same target, identified risks/ATE and uncertainty	1
Robustness and cost	Two refuters, run stability and fit/query runtime	1
Conclusions and Q&A	Evidence, identification limits and defense	1

Application: CLO3, CLO4. Presentation total: $4 + 6 = 10$ points.

Presentation: notebook and demonstration

Submit slides and a Colab-compatible notebook with saved outputs, setup instructions and a table of contents.
Include the UCI source, timing, preprocessing, splits, graph constraints, fitted CPDs and identification output.
Explain the exact differences between B0, H1 and H2. Save neural settings, checkpoints and learned graph outputs.
Show Tables A–C: TEST prediction, common-reference intervention estimates, uncertainty, robustness and runtime. See slides 74–75.
Demonstrate one intervention contrast on UCI and show two supported DoWhy refutation checks. Explain identification or overlap failures.
Include package versions, seeds, citations and AI disclosure. Synthetic-data experiments and A1–A3 are not required.

Report: introduction and problem (3 points)

2.1 Introduction (1 point, CLO1, CLO2): state the UCI question and why a basic CBN is the reference for testing DL extensions.
2.2 Related work (1 point, CLO1, CLO2): review CBNs and DL methods relevant to the selected causal stages.
2.3 Problem statement (1 point, CLO1, CLO2, CLO4): define the population, variables, timing, DAG and estimand.
Example target: P(Y = 1 | do(X = x)) or E[Y | do(X = x1)] minus E[Y | do(X = x0)].
Distinguish causal discovery, formal identification and statistical estimation. State which assumptions each extension requires.

Report: methods (2 points)

2.4 Proposed methods (2 points, CLO3, CLO4).
Specify B0: traditional DAG learning with pgmpy or bnlearn, domain constraints, non-neural CPDs and identified intervention formula.
Specify H1: selected neural CPDs on the fixed B0 DAG, neural architecture, loss, regularization and training.
Specify H2: neural discovery method, the same domain constraints as B0, graph threshold, convergence, identification and non-neural CPD fitting.
Specify the shared prediction protocol, identified intervention formulas and reference-cohort standardization for B0/H1/H2.
Describe uncertainty estimation, refuter compatibility, reproducibility and failures. See slides 59–62, 68–71 and 74–75.

Report: experimental design (1 point)

2.5 Experimental design (1 point, CLO3, CLO4).
Document UCI eligibility, timing and overlap. Predeclare the test cohort and baseline reference covariates.
Fit preprocessing, DAGs and mechanisms on TRAIN. Use VALIDATION for choices, then freeze settings before TEST evaluation.
Use the same TEST rows for prediction scores and reference-covariate standardization of each identified do-query. Do not use test Y to compute effects.
Run at least five paired B0/H1/H2 comparisons. Separate prediction metrics, effect estimates, robustness, uncertainty and runtime.
Specify two DoWhy refuters, compatibility limits and expected behavior. See slides 28, 59–62, 68–71 and 74–75.

Report: results and conclusions (2 points)

2.6 Experimental results (1 point, CLO3, CLO4).
Compare B0/H1/H2 on test prediction, test-reference intervention means and ATE, uncertainty, robustness and runtime. Report Tables A–C.
On UCI, quantify paired predictive and effect differences, uncertainty and sensitivity under stated assumptions. No synthetic verification is required.
2.7 Conclusions (1 point, CLO4, CLO5).
Explain whether neural mechanisms or graph learning improve prediction or stability, and whether the intervention conclusion changes.
A lower test prediction loss supports better prediction. Stable effects or narrow intervals alone do not prove causal accuracy. Discuss limitations and null findings.

Report: abstract and defense (2 points)

2.8 Abstract (1 point, CLO4, CLO5). State the UCI question, basic CBN baseline, DL extensions, main predictive/intervention comparisons and causal limitations.
2.9 Questions and answers (1 point, CLO1–CLO5). Defend the DAG, identification, DL integration, comparison fairness and refutation interpretation.
Submit a .docx or a PDF compiled from LaTeX, plus source files when requested.
Keep dataset versions, outcome definitions, splits, DAGs and results consistent across the report, notebook and presentation.
The following slide consolidates the report rubric.

Report rubric: 10 points

Section	Points	CLO mapping
2.1 Introduction	1	CLO1, CLO2
2.2 Related work	1	CLO1, CLO2
2.3 Quantitative problem statement	1	CLO1, CLO2, CLO4
2.4 Methods and model definitions	2	CLO3, CLO4
2.5 Fair comparison and experiment design	1	CLO3, CLO4
2.6 Prediction and intervention results	1	CLO3, CLO4
2.7 Conclusions	1	CLO4, CLO5
2.8 Abstract	1	CLO4, CLO5
2.9 Questions and answers	1	CLO1–CLO5

Total $= 10$ points. Full credit requires the evidence on slides 45–49. Award partial credit within each section.

Reproducibility and interpretation checklist

Use one UCI dataset with documented variables, timing, treatment/control values and a common test reference cohort.
Implement B0, H1 and H2 as defined. Synthetic-data experiments and additional ablations are not required.
Fit on TRAIN, select on VALIDATION and freeze settings before TEST. Use at least five paired runs with shared splits.
Score predictions against test Y. Compute standardized intervention means from trained models and common test baseline covariates.
Report 95% intervals with their method, paired-run spread, two refutation checks and runtime.
Preserve raw results and configurations.
Use Tables A–C on slides 74–75. Keep predictive and causal claims distinct; report unsupported profiles, failures and AI assistance.

References and API documentation

DoWhy documentation: identification, estimation and refutation
https://www.pywhy.org/dowhy/v0.14/
DoWhy: simulating the impact of interventions
https://www.pywhy.org/dowhy/v0.14/user_guide/causal_tasks/what_if/interventions.html
pgmpy: DiscreteBayesianNetwork API
https://pgmpy.org/api/generated/models/pgmpy.models.DiscreteBayesianNetwork.html
Sharma & Kiciman (2020). DoWhy: An End-to-End Library for Causal Inference. arXiv:2011.04216.
The tutoring DAG, CPDs and reference values are a synthetic teaching example.
UCI repository: https://archive.ics.uci.edu/
Student Performance (ID 320): https://archive.ics.uci.edu/dataset/320/student+performance

Implementation guide: required experiment map

Required models: B0 uses traditional DAG learning and basic CPDs; H1 changes selected CPDs to neural models; H2 uses neural DAG learning.

Required data: one UCI tabular application with the same TRAIN/VALIDATION/TEST rows and reference population for all models.

Required comparison: prediction scores, intervention means and ATE with uncertainty, paired-run stability, refutation and computational cost.

The tutoring walkthrough explains concepts only. The project has no synthetic-data generation or synthetic verification requirement.

A1–A3, combined H1+H2 models and extra GCM simulation are optional extensions, without separate mandatory rubric items.

Slides 54–63 cover models. Slides 68–71 and 74–75 specify core evaluation. Slides 76–80 cover submission and optional extensions.

Causal question: a specification before modeling

Write one sentence naming the population, treatment T, outcome Y and the time of intervention.

Specify control and treatment values, outcome measurement time and the target effect, such as ATE = E[Y under do(T=1)] minus E[Y under do(T=0)].

Create a variable dictionary: meaning, coding, measurement time, proposed causal role and missingness for every selected variable.

List candidate confounders measured before T. Explain the role of any mediator or collider before deciding how to use it.

State assumptions about unmeasured confounding, consistency and overlap. Separate documented facts from assumptions.

Deliverables: the question, dictionary, target formula and an assumptions table shared by the notebook, slides and report.

UCI data: source, timing and eligibility audit

Record the UCI URL, dataset ID or DOI, download date, license, subject file and file checksum. Preserve the original downloaded data.

Report row counts before and after every exclusion. Check duplicates, units, category codes, missing values and class balance.

For Student Performance, define pass from G3, then remove G3 from predictors. Document which subject and sampling unit you use.

Use G1 or G2 only if their measurement precedes the chosen treatment. Explain whether the same student appears more than once.

Inspect treatment counts and outcome counts within important covariate groups. Flag sparse groups and weak overlap.

Deliverables: a source record, eligibility flow, variable-timing table and data-quality summary.

Shared preprocessing and split construction

Create common TRAIN, VALIDATION and TEST indices before fitting. A 70%/15%/15% split is an example. Use group or time splits when appropriate.

Fit imputation, encoding, scaling and discretization on TRAIN only. Keep measured variables and bin definitions comparable across models.

Use VALIDATION for architecture, graph thresholds, regularization and stopping. Save all selected settings before evaluating TEST.

Predefine baseline reference covariates C, measured before treatment. They must retain their baseline role under every admissible DAG.

Use the same TEST row IDs for predictive scoring and causal standardization. Y_test scores prediction. C_test defines the causal reference population.

Save indices, transformations and support rules. Do not select graphs or retune models using test results. See slides 59–62 and 68.

DAG and identification: required checks

For the UCI B0, learn a candidate DAG on TRAIN using a traditional method: pgmpy HillClimbSearch, PC or GES, or bnlearn (R) hc or tabu.

Before learning, define forbidden or required edges and temporal ordering from domain knowledge and cited literature. Declare assumptions.

Record the algorithm, score or independence test, settings, constraints and node order. Explain unresolved directions or any graph repair.

H1 uses the exact B0 DAG for each paired run. H2 learns a DAG with a neural method using the same variables, train rows and domain constraints.

Run DoWhy identification for each DAG. A learned graph or a software-identified effect still requires defensible causal assumptions.

ChatGPT may suggest graph constraints, but verify them independently and disclose AI use. The supplied tutoring graph is a classroom illustration.

Tutoring DAG learning: constraints and edge weights

Suppose a traditional learner searches for edges among Z, T and Y. Timing knowledge rules out an exam result causing earlier preparation.

In R bnlearn, a blacklist can forbid Y to Z. In pgmpy, expert knowledge can encode forbidden edges and time order.

A score-based learner compares candidate graphs using a declared score. A test-based learner uses conditional-independence evidence.

A neural learner optimizes a parameterized mechanism and graph representation. An adjacency mask can keep forbidden weights at zero.

An edge weight is an internal learning quantity. It is not automatically a causal effect or the probability that an edge is true.

If data leave directions unresolved, report the ambiguity and any additional assumptions. Apply the same prior timing information to B0 and H2.

B0: basic CBN implementation workflow

1. Learn the UCI B0 DAG on TRAIN using pgmpy or bnlearn and domain constraints. Select settings on VALIDATION, then freeze the graph and settings.

2. Declare variable states. Fit and check non-neural CPDs on TRAIN, including state order, normalization and prior settings.

3. On TEST, predict Y from eligible evidence and compare with observed Y. Use the same prediction inputs and test rows for every model.

4. For each TEST baseline profile C_i, compute the identified mean q_m(t,C_i) under do(T=t) from the trained model, for both treatment values.

5. Average over the same TEST profiles: risk_m(t) = mean_i q_m(t,C_i). ATE_m = risk_m(1) minus risk_m(0). Do not use test Y for this calculation.

6. Compare B0/H1/H2 using the criteria on slide 69 and Tables A–C on slides 74–75. The concrete standardization example appears on slide 61.

H1: neural CPDs on the fixed B0 graph

For each paired run, copy the B0 DAG. Replace one CPD, preferably Y. Keep states, splits and all other CPDs fixed.

Use only that node’s parents as neural inputs. For binary Y, output a probability and train with binary cross-entropy.

Record architecture, dropout, optimizer, learning rate, batch size and parameter count. Fit weights on TRAIN and select stopping on VALIDATION.

Enumerate the discrete parent configurations to form a normalized neural CPD table. Insert it into a copy of B0 and check the resulting model.

After freezing H1, evaluate prediction on the common TEST rows. Standardize its do queries over the same baseline profiles as B0 and H2.

Compare prediction, standardized effects, stability and runtime. The concrete calculation appears on slide 61.

H1: a concrete intervention calculation

For the tutoring DAG, fit B0 and neural H1 on TRAIN. Let Z_i be the baseline preparation of each person in the common TEST reference cohort.

For model m, compute q_m(t,Z_i) = P_m(Y=1 | t,Z_i). This equals the conditional intervention risk here because Z is a valid backdoor set.

Compute risk_m(t) = (1/n_test) sum_i q_m(t,Z_i), for t=0 and t=1. Then ATE_m = risk_m(1) minus risk_m(0).

Equivalently, weight each q_m(t,z) by the same empirical TEST frequency of z. TEST outcome labels do not enter this estimate.

B0 and H1 use the same reference weights and DAG, changing only the selected CPD. For H2, derive its own valid formula for the same target cohort.

For other graphs, use identified adjustment or conditional truncated factorization and integrate mediators. A direct model-marginal do query is a separate check.

H2: neural causal discovery workflow

Choose a neural structure learner, such as DAG-GNN. Use the same variables, TRAIN rows and domain constraints as the traditional B0 learner.

Declare the loss, sparsity settings, edge threshold, tuning budget and convergence checks. Select settings on VALIDATION.

Save adjacency weights and the final DAG. Record thresholding, cycle repair and uncertain directions. Freeze choices before TEST evaluation.

Rerun identification. Fit non-neural CPDs on TRAIN with the B0 estimator family and prior strength.

Evaluate prediction and identified intervention means on the same TEST cohort as B0/H1. Baseline reference covariates must remain pre-treatment.

Report unsupported or unidentified queries explicitly. Compare effects, uncertainty, stability and runtime. See slides 61 and 68–75.

H2: domain constraints and adjacency masks

List forbidden edges with reasons. Example: a final exam result Y cannot cause prior preparation Z.

Define a fixed convention, such as W[i,j] meaning edge i to j. Build an allowed-edge mask M with zeros for forbidden edges.

Use effective adjacency W_eff = M multiplied elementwise by W in the model and its graph constraints. Keep diagonal entries zero.

If the library offers a forbidden-edge interface, use it. Otherwise document and verify the masking changes to the implementation.

Keep the acyclicity constraint active. Verify that forbidden weights remain zero and the final graph is acyclic.

Save the constraint list, mask, node order, learned weights and a constraint-compliance check.

Optional: A1: regularization ablation from H1

Parent: the full H1 model. First choose an H1 regularizer that is active, such as dropout with a declared nonzero rate.

Single change: set that dropout rate to zero. If H1 uses another regularizer, remove exactly that one penalty instead.

Keep the DAG, replaced node, layer widths, activation functions, inputs, data splits and other training settings fixed.

Keep the same validation stopping rule and maximum budget. Do not retune other hyperparameters for the primary ablation.

Across paired runs, compare training and validation loss, held-out prediction and the estimated causal effect.

H1 versus B0 already tests neural removal when H1 has one neural CPD. A duplicate B0 run is not a new ablation.

Optional: A2: one removed domain constraint from H2

Parent: full H2, which forbids Y to Z because Y occurs later than Z. Select this one prohibition before inspecting results.

Single change: allow that candidate edge by changing its mask entry from zero to one. Keep all other constraints, including acyclicity.

Retrain graph learning with the same split, seed, threshold and tuning settings. Removing a ban does not force the edge to appear.

Compare adjacency weights, learned edges and direction stability. Then rerun identification and fit the same non-neural CPD family.

Report effect changes only when interpretable. Label a graph contradicting time order as a sensitivity result.

If the DAG stays unchanged, report that null finding. If no removable constraint exists, revise the H2 design before running A2.

Optional: A3: smoothing versus maximum likelihood

Parent: discrete B0 with smoothed CPDs. Single change: estimate CPDs by maximum likelihood from the same training rows.

Keep the DAG, node states, bins, missing-data treatment, queries and inference procedure fixed. Refit parameters only.

Example group T=1, Z=0: five students, five passes and zero failures. The maximum likelihood pass probability is 5/5 = 1.

With an illustrative add-one prior for each outcome, the smoothed probability is (5+1)/(5+2) = 6/7, about 0.857.

This add-one example explains smoothing. BDeu distributes its declared prior strength across states and parent configurations.

Compare sparse-cell probabilities, predictive calibration, effect estimates and variability. Improvement in either direction is possible.

Optional: A3: pgmpy fitting pattern and checks

Create two fresh DiscreteBayesianNetwork objects with identical edges and nodes. Use one shared state_names mapping.

B0 fit: estimator=BayesianEstimator, prior_type="BDeu", equivalent_sample_size=5 as an example declared setting.

A3 fit: estimator=MaximumLikelihoodEstimator. Use the same training data and state_names. Omit the prior arguments.

Run check_model() for both. Compare CPD dimensions, state order, normalization, zero entries and unsupported parent configurations.

Recompute the same prediction and do queries for both fitted models. Count any infinite log loss from zero-probability predictions.

Report numerical stabilization separately from probability smoothing. Preserve the raw CPDs and disclose scoring conventions.

Paired runs, tuning and uncertainty

Use at least five paired runs. Within each run, B0, H1 and H2 share TRAIN/VALIDATION/TEST indices and the test reference cohort.

Compare H1 with B0 and H2 with B0 within each run. Report prediction scores, intervention means, ATE and signed paired differences.

Summarize run variability with mean and standard deviation. If splits change, report that both training and reference-cohort composition can vary.

For training-seed stability alone, keep the split and TEST reference rows fixed. This variability is not a population-ATE confidence interval.

Declare an uncertainty method: conditional on fixed TEST covariates, or including sampling of the reference population. Refit components as required.

Separate fixed-DAG from graph-learning uncertainty. Narrower intervals alone do not prove accuracy. Record timing and hardware.

Core evaluation criteria: prediction and intervention

Prediction on TEST: AUROC and macro-F1 plus Brier score or log loss for binary Y. For continuous Y, use MAE and RMSE.

Intervention on the common TEST reference: mean outcome under each treatment value, ATE, and a 95% confidence interval with a declared method.

Identification and support: report the graph, formula or adjustment set, assumptions, treatment overlap and unsupported reference profiles.

Robustness: report two DoWhy checks, such as placebo treatment and data subset. Explain compatibility with the estimator being checked.

Stability and cost: use at least five paired runs, report mean and standard deviation, signed model differences, fit/query time and hardware.

Interpret jointly. Prediction can be scored against observed outcomes. Intervention comparison assesses estimates and robustness under stated assumptions.

DoWhy refutation: execution and interpretation

Run at least two checks, including a permuted-treatment placebo and a data-subset refuter. Record simulation counts and seeds.

For the placebo, state that permuting treatment should remove the original treatment association. Inspect the returned effect and test definition.

For the subset check, compare estimates after repeated subsampling. Explain the subset fraction and expected variability.

Run the checks on the supported DoWhy estimate. State which estimator and graph each result actually validates.

For a custom neural estimate, verify refitter compatibility. Otherwise implement and label an analogous retraining check separately.

Retain the required DoWhy outputs for a supported reference estimate. Passing checks does not prove the DAG or absence of confounding.

Tutoring refutation: placebo and subset examples

Placebo treatment: shuffle the tutoring labels in a refutation experiment, refit the supported estimator and compute its effect again.

Shuffling breaks the original treatment assignment relationship. A placebo effect near zero is an expected diagnostic behavior, subject to finite-sample variation.

Subset check: repeat estimation on, for example, 80% training subsets and inspect how much the effect changes.

A large shift can signal sparse groups, unstable fitting or sensitivity to sample composition. It does not identify the cause by itself.

State which graph and estimator each DoWhy output uses. A linear-estimator refutation does not automatically validate the neural H1 pipeline.

For a neural custom check, refit its preprocessing and weights correctly, preserve the target definition and label the procedure separately.

Optional intervention simulation checks

Optional extension: use DoWhy GCM to examine intervention simulation. Record its mechanisms and compare them with the fitted CBN mechanisms.

For a population-marginal check, fix T=t in every generated row. Estimate Y risk or mean and inspect Monte Carlo variability.

For the UCI comparison, match the same TEST baseline reference profiles used by the exact query. Use valid conditional simulation under the intervention.

Simply drawing a new baseline population does not reproduce test-reference standardization. Keep the two targets clearly labeled.

Match mechanisms where feasible. Otherwise explain that mechanism differences and simulation error can both change the results.

Use validated neural probabilities or tested custom mechanisms. Individual counterfactuals need stronger structural assumptions and remain optional.

Tutoring simulation, do queries and counterfactuals

A do-query asks for a distribution, such as the average pass probability if all target students receive tutoring.

Simulation approximates that distribution by generating many outcomes from the fitted causal mechanisms with T fixed to the intervention value.

Example: an exact risk of 0.64 and a simulation estimate of 0.638 can differ because simulation uses finite random draws. These values are illustrative.

For the required comparison, simulation must match the same baseline test profiles. Drawing a new preparation mix changes the reference population.

An individual counterfactual asks whether one particular student who failed without tutoring would have passed with tutoring.

That individual question needs additional structural and noise assumptions. A probability table for population risks does not determine the answer by itself.

Table A: standard prediction comparison

Rows: B0, H1 and H2 for each paired run, plus a summary row per model. Use the same test observations and prediction-time information.

Binary outcomes: AUROC, macro-F1 and Brier score or log loss. Declare the F1 threshold using validation data and report class balance.

Continuous outcomes: MAE and RMSE. Preserve the same outcome definition and units across models.

Report individual runs and mean with standard deviation. Add signed paired H1-minus-B0 and H2-minus-B0 differences.

Record test size, split/seed, graph ID, fit time, prediction time and hardware. Report undefined metrics or failed runs with a reason.

Table B covers intervention estimates. Table C covers robustness and computational cost. See slide 75.

Tables B and C: interventions, robustness and cost

Table B fields: model/run, common reference-cohort ID and size, treatment/control values, identified formula and adjustment set where applicable.

Report mean outcome under each intervention, ATE, 95% confidence interval, interval method and graph/overlap status.

Table C fields: placebo result, subset result, paired-run effect spread, signed differences against B0, fit/query times and hardware.

Report raw runs and model summaries. Keep the same reference cohort within a run. Label any graph-specific identification failure.

Discuss whether prediction, estimated intervention conclusions, stability or cost differ. Do not rank causal accuracy by effect size or interval width alone.

No true-effect error, synthetic benchmark or mandatory ablation table is required. Optional extension results may appear in an appendix.

Notebook structure and reproducible artifacts

Sections: setup, data audit, split creation, DAG and identify, B0, H1, H2, prediction evaluation, intervention evaluation, refutation and Tables A–C.

Use one configuration object for seeds, data paths, node order, state names, priors, neural settings, masks and graph thresholds.

Save split indices, transformations, CPDs, neural checkpoints, adjacency weights, final graphs and identification text.

Save raw per-run results and tables in machine-readable files. Record exact package versions and the repository revision.

Restart a fresh Colab runtime and execute all cells in order. Include commands for setup and a declared expected runtime.

Retain saved outputs in the submitted notebook. Document downloads, external code and any manual steps.

Presentation demonstration: concrete sequence

1. State the UCI question, timing, graphs and assumptions. Show the common TEST reference cohort and treatment/control values.

2. Explain which data fit the model (TRAIN), select it (VALIDATION) and support final prediction/reference standardization (TEST).

3. Show the H1 CPD replacement and the H2 learned graph, constraints and identification output.

4. For several test baseline profiles, display q_m(0,C_i) and q_m(1,C_i). Average over all common reference rows to obtain each model's risks and ATE.

5. Compare test prediction, standardized effects, uncertainty, stability, runtime and DoWhy refutation outputs.

6. Explain a null finding or failure and the limit of the causal claim. Trace every displayed result to its configuration and saved output.

Report sections: evidence for each requirement

2.1–2.3: motivate the UCI question, review CBN and selected neural methods, then define variables, timing, DAG and estimand.

2.4: specify B0/H1/H2 mathematically, give neural and graph-learning settings, and explain their controlled differences.

2.5: define TRAIN/VALIDATION/TEST use, common test-reference standardization, paired runs, uncertainty and refutation.

2.6: compare TEST prediction and standardized effects, uncertainty, robustness, failures and runtime. See slides 61, 68–71 and 74–75.

2.7–2.8: conclude from the measured results and summarize the main finding with causal limitations. Include null findings.

2.9: defend modeling and evaluation choices. Keep citations, AI disclosure and all definitions consistent across submissions.

Common failures and required responses

No overlap or unsupported cells: report affected groups, quantify the problem and narrow the causal target only with justification.

Unidentified or temporally implausible H2 graph: report it explicitly. Avoid presenting a numerical result as a validated causal effect.

Cycles after thresholding: apply a declared repair rule consistently or report graph-learning failure. Save both raw and final graphs.

Zero ML probabilities or absent parent states: document the estimator convention. Keep any metric clipping separate from CPD smoothing.

Neural overfitting, unstable effects or poor test prediction: show losses and paired runs. Do not suppress unfavorable results.

A failed package integration or refuter: provide the actual limitation, a supported reference check and a clearly labeled alternative.

Optional extensions and implementation references

Optional H1+H2: use neural CPDs on the learned H2 graph. Compare with both parents to separate graph and mechanism contributions.

Other optional DL roles include conditional density models, learned representations and approximate inference. State additional assumptions.

DAG-GNN paper and code: arxiv.org/abs/1904.10098 and github.com/fishmoon1234/DAG-GNN.

Neural NOTEARS alternative: github.com/xunzheng/notears. Use its nonlinear neural implementation if selecting this option.

Traditional discovery: pgmpy.org/guides/causal_discovery.html and bnlearn.com. Declare the R bnlearn version and any R/Python bridge.

Required core: the UCI B0/H1/H2 comparison with standard prediction, intervention, uncertainty, robustness and runtime reporting. A1–A3 are optional.